DESIGN AND IMPLEMENTATION OF AN EMOTION-AWARE VOICE ASSISTANT

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

Submitted by

Rohit Meena (24U021015 V Semester)
Vishal Chaudhary (24U021019 V Semester)

Under the supervision of

Dr. Amit Kumar Nandanwar
Assistant Professor
(Computer Science and Engineering)

November 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL **(M.P.)**



CERTIFICATE

This is to certify that the work embodied in this report entitled **"**Design and Implementation of an Emotion-Aware Voice Assistant**"** has been satisfactorily completed by Rohit Meena(24U021015) and Vishal Chaudhary(24U021019). It is a bonafide piece of work, carried out under our guidance in the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-27**.**

Dr. Amit Kumar Nandanwar,
Assistant Professor,
Department of CSE,
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia,
Head/Department Coordinator,
Department of CSE,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF
INFORMATION TECHNOLOGY
BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project report entitled "Design and Implementation of an Emotion-Aware Voice Assistant" presented here is in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of Dr. Amit Kumar Nandanwar. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Rohit Meena(24U021015)
Vishal Chaudhary(24U021019)

ACKNOWLEDGEMENT

We, Rohit Meena and Vishal Chaudhary, would like to thank to everyone who helped and directed us in successful completion of our minor project titled "Design and Implementation of an Emotion-Aware Voice Assistant".

First of all we would like to thank our esteemed Director, Prof. Dr. Ashutosh Kumar Singh for giving us the opportunity to undertake our project by giving us the resource and facilities for doing the project. His vision and enthusiasm has kept us motivated through out the completion of the project.

We are highly thankful to our Project Guide, Dr. Amit Kumar Nandanwar for his constant guidance, valuable suggestions, and expert advice throughout the project. We, grateful to his knowledge and inputs, which enabled us to make our project more effective and we are motivated by his support.

We would like to thank our Department Coordinator Dr. Rahul Chaurasia, for the support and encouragement given while working on this project.

Besides, we would like to thank all the faculties of the CSE for their precious guidance and support in clarifying our doubts and providing the right direction.

Thanks to our friends for their constant support and suggestions, and helping us through many stages of this project. Their suggestions and support kept us engaged and motivated throughout the project work.

And finally, and not the least we would like to thank our parents and families for their love, patience and encouragement and for the moral support throughout the project. Their understanding towards us and support were the prime motivator to what we are today.

We thank you all dearly for all the help that you gave us directly and indirectly to make this project a success. Without your help and support this project would not have taken place.

Rohit Meena[24U021015]
Vishal Chaudhary[24U021019]

AREA OF WORK

This study is situated at the intersection of speech processing, dialogue systems and emotion-conscious human-computer interaction. It comprises several components, such as speech recognition, emotional signals and responses, voice synthesis and interruption management in a browser-enabled assistant.

- Interaction in speech and language translation as text translation as speech transcription dialogue as text dialogue as speech.

- Emotion processing has transcript data and maybe an optional pretrained acoustic classifier for matching compatible WAV input.

- The signals and the reply – a short reply based on instructions – are transmitted and received by the language model.

- These include a mixture of Fast API routes, web browser audio, session state, captions, playback, and cancellation.

- Assessment includes pilots of SOVA tasks along with counting, formulas and case level results preservation.

The project makes use of pretrained models and services hosted in the cloud, and has not developed any new speech, language or emotion models. The project probably will produce an end-to-end demonstrator available for viewing, and a schedule of small, pilot studies.

TABLE OF CONTENT

S.no	Title	PageNo.
	Certificate	
	Declaration	
	Abstract	
1	Introduction	2
2	Literature review or Survey	4
3	Methodology & Work Description	11
4	Proposed algorithm	15
5	Proposed flowchart/ DFD/ Block Diagram	16
6	Tools & Technology Used	20
7	Implementation & Coding	20
8	Result Analysis	25
9	Conclusion & Future Scope	30
10	References	31

LIST OF FIGURES

Fig.	Description	Page No.
Figure 4.1	Emotion-signal fusion and response policy	12
Figure 6.1	Turn-based Emotional voice assistant AI architecture	16
Figure 6.2	Pilot evaluation and evidence flow	17

LIST OF TABLES

Table No.	Description	Page No.
Table 1	Evaluation frameworks relevant to this project	4
Table 2	Implemented components and roles	20
Table 3	Emotion aware voice assistant pilot metrics and denominators	25
Table 4(a)	Published recognition and understanding values	26
Table 4(b)	Published generation values	26
Table 5	SOVA-to-local task mapping Calculation and evidence file map	27

ABSTRACT

This project implements a browser-based emotion-aware voice assistant that accepts speech or text, displays the transcript, estimates an emotion cue, generates a reply, and returns speech. The FastAPI backend uses Groq Whisper for speech recognition and Groq GPT-OSS 120B for response generation. Emotion handling combines transcript rules with an optional pretrained SUPERB/Wav2Vec2 classifier for compatible WAV audio. ElevenLabs provides speech output. The models were used as supplied; no fine-tuning was performed.

A limited, adapted evaluation covered 1,220 cases: 1,000 LibriSpeech utterances (3.55% corpus WER; 1,000/1,000 requests completed) and 55 cases each from TriviaQA, LibriSQA, Spoken SQuAD, and Alpaca. Results were 35.58% token F1 and 0% exact-match proxy on TriviaQA; 74.55% LibriSQA accuracy across 55 cases (78.85% among 52 valid options); 25.54% token F1 and 0% exact-match proxy on Spoken SQuAD; and 20.93% ROUGE-L F1 with 26.66% token F1 on Alpaca. These adapted pilots are not a full SOVA-Bench reproduction. Emotion classification accuracy and human-rated speech quality were not measured.

CHAPTER 1. INTRODUCTION

1.1 Background

In order to provide a useful response with the correct intent the voice assistant should produce an appropriate response. When using speech there is some uncertainty due to accents, background noises, speed of speech and quality of recording. The response could be affected by emotional signals, however labelling is only useful if the source and certainty of the signal

is considered. This project takes the key components and puts them into an inspectable web application.

1.2 Project Context

The application to be audited is a web demo, using FastAPI [26] to serve. Front-end and API are incorporated into one application.
Speech is converted to text via a speech recognition service. Text responses are sent directly into the dialogue flow. Emotional signals are generated based on the transcript information. The source is shown in the interface. The chosen emotional signal and response strategy will help determine the prompt given to the language model. The language model will not be trained.
Captions are dependent on the browser supporting SpeechRecognition [30]. The transcript will be used as the definitive input to the backend.

1.3 Motivation

If there are errors in the speech to text, inappropriate tone of the response, inability to play the audio or the assistant continuing to talk then users will lose confidence in the product. This is where the demo has been created to show the transcript, source of emotional signal, response text, ability to play and cancellation. SOVA-Bench provides motivation to evaluate each aspect of the project: recognition, understanding, emotional signals and generation [1].
1.4 Report Structure
This report will first look at related systems and benchmarks (chapter 2). Problem definition, methodology, algorithm, architecture and implementation will be described (chapters 3-7). Calculations will be provided for the local pilot (chapter 8) and the same results shown in the paper. Differences in the protocols will also be described. Finally a summary of the work will be provided (chapter 9).

1.4 Interaction Context

Voice assistants have a different interaction context compared to traditional text applications. With a text interface there is the ability to read through what the user has written before they send their request. Then they can read their response as slowly as they want. However, using voice interaction, the system will receive a signal from the user that may include pauses, varied pronunciations, background noise, varying speeds and incomplete sentences. The user would expect a smooth response. This means that the whole interaction process is important to consider. From capturing sound from the microphone, converting it into text, generating a response, converting the text into voice and finally playing it back so that a new turn can be started.

This project has separated out all aspects of processing the voice turn. This means we can see exactly what was received, the text generated by the speech recognition, the emotion source selected, what response was generated and if the speech was fully played. This application also allows text to be entered so the response generation and emotion policy can be seen without using speech recognition.

When using an emotion aware system, the emotional label is an estimation and will not be a true reflection of how someone is feeling. Using appropriate word choices can give some useful context. If available, using vocal characteristics can help to give more evidence. However both of these can be incomplete and misleading. This project shows where the cue came from and uses this to help decide the style of response.

1.5 System Boundaries

The browser handles all the user interaction. This includes capturing the microphone input, displaying the captions and response text, playing the generated audio, showing what state the assistant is in and monitoring any local activity to see if there was an interruption. The backend handles the authenticated sessions and voice processing. Speech recognition, language generation and speech synthesis is done using configured providers. This helps as each step can be looked at individually if an issue occurs with a request.

It also means we know what we are not in control of. Whilst we can validate requests and show errors from the providers, we cannot ensure that the remote provider will always be available. There will always be a chance of errors due to the recording conditions. We can ask to cancel the current turn however this will depend on when the request is made and the audio being played from the browser.

CHAPTER 2. LITERATURE REVIEW

2.1 SOVA-Bench as the evaluation reference

SOVA-Bench [1] is a benchmark used to measure Voice Assistants in terms of knowledge, speech question answering, speech recognition, paralinguistic skills and speech generation. It provides separate scores for individual tasks for each of these categories as shown in Table 4 [1]. These have been used as initial categorizations for this research project and also indicate aspects not targeted by the current tests.

Here, the six models (Mini-Omni, Mini-Omni2, Moshi, LLaMA-Omni, Freeze-Omni and GLM-4-Voice) and one cascaded reference system are compared. Due to the test set variation, comparable evaluation must be performed by using similar metrics and pipeline [1]-[4][15][16].

Additionally, the models were evaluated across various datasets such as TriviaQA, Libri Speech, LibriSQA, Spoken SQuAD, IEMOCAP, Common Voice and Alpaca [1]. Interestingly, very little data was used for certain pilots, which might have affected our ability to reproduce the results published in the paper.

2.2 Related evaluation frameworks

Table 1. Evaluation frameworks relevant to this project.

Work	Emphasis	Connection to this project
SOVA-Bench [1]	Knowledge, recognition, spoken understanding, paralinguistic understanding, and generation	Main reporting reference; local runs are adapted pilots.
Dynamic-SUPERB [5], [6]	Broad speech-task benchmark suites	Motivates task-specific reporting.
AudioBench [7]; AIR-Bench [8]	Audio understanding	Context for evaluating audio-language systems.
VoiceBench [9]	Voice-assistant ability	Evaluation beyond speech recognition alone.
IEMOCAP [19]	Emotional dyadic speech	Research context; not used to measure Emotion Aware Voice Assistant accuracy.
LibriSpeech, TriviaQA, Common Voice [21]–[23]	Recognition, knowledge QA, and speech attributes	Only the subsets stated in Chapter 8 were evaluated locally.

2.3 Past Systems and Project Position

2.3.1 Speech-Interactive Systems

Speech-interactive systems use both streaming and token-based approaches. SOVA-Bench compares these systems with a cascaded reference [1]. Examples include Mini-Omni, LLaMA-Omni, and Moshi [2]–[4]. Dynamic-SUPERB provides related speech-benchmark context [5], [6]. Other speech-language models discussed in the literature include SALMONN, SpeechGPT, and Qwen2-Audio [10]–[12]. HuBERT is a speech-representation model [13], and HiFi-GAN is a vocoder [14]. Freeze-Omni and Mini-Omni2 explore low-latency or duplex interaction [15], [16].

A cascaded assistant connects speech recognition, language generation, and speech synthesis as separate stages. The Emotion Aware Voice Assistant follows this modular pattern, making stage outputs visible during debugging and helping us trace errors. It still depends on external providers, so service availability and response latency can vary. The implementation uses FastAPI, Groq, and ElevenLabs services [26]–[28].

2.4 Emotion-Assisted Response Generation

Emotion can be realized in speech through acoustic features or through linguistic features and context. The IEMOCAP database [19] is an excellent database to use for emotion detection research in speech. This project uses a trained acoustic classifier, if the corresponding audio is available, otherwise the transcript rules, used to determine the emotion label. This pair of label/source is then sent to the response policy. In the fact there is no labeled holdout this project can be sure of emotion accuracy.

2.5 The research gap and project position

This work partially fills the research gap by showing an implementation and a report on transcript, emotion source, answer generation, speech output and interruption paths. It offers a cascaded system within an integrated package, with a handful of traceable initial steps. On the other hand, it has not built new baseline models with state-of-the-art performances. It is worth mentioning, SOVA-Bench is the reporting tool used to show the variations of dataset size, modality, model, evaluation metric and the like. Local values have been used.

2.6 Cascading and End-to-End Voice Architectures

The speech models can be arranged in one of two ways, either cascade, where each step happens in series, or end-to-end, where everything is done in parallel. If a cascade arrangement is used there will usually be three main modules: speech-to-text [27], the language model, and the text-to-speech [28] system. This project will use a cascade arrangement. SOVA-Bench has published performance comparison of many architectures for speech focused models [1].

Using a cascading set-up offers multiple benefits for this project. Because we know what each service is supposed to do in the chain we can be selective about what is re-reviewed. For instance, if the transcription looks suspicious, we can double check it against a reference. If the output looks wrong, we can double check the language model output. If speech synthesis is broken, the model can double check if the response is rendered correctly.

Service dependencies may cause failures: Transcribed speech can incorrectly change the meaning, but recognition may take extra time. Synthesis may cause some delays, and you may experience failures when requesting services due to bad external services. While the cascaded structure can help identify errors, errors can be caused by previous stages, which can have a significant effect on the model.

End-to-end speech models try to overcome those obstacles by combining various speech and language tasks. Past research on Mini-Omni, Mini-Omni2, Moshi, LLaMA-Omni and Freeze-Omni [2]-[4], [15]-[16] show models with different interfaces and out-of-model dependencies. This project uses a modular external service approach instead of an end-to-end locally trained model.

2.7 Emotional Information and Uncertainty

Emotion can also be transmitted using lexical features, which may be observed in transcripts by words that directly indicate emotion, such as anger, worry, thankfulness and joy. Acoustic features may be a mixture of pitch, energy, time and voice quality. Experimental corpora and models for exploring speech representations like IEMOCAP or HuBERT have been developed [13], [19]. Although a feature or classifier output may be available, there is no guarantee that emotion can be reliably extracted.

This implementation uses a rules-based transcription algorithm and an optional pre-trained acoustic classifier, which each have their own challenges: Keywords can cue explicit language, but can't cue implied or sarcastic language or context. The acoustic classifier evaluates properties of a waveform but can be cued or thrown off by input that affects recording quality, language, accent, or the domain of the training data. If the model isn't available or the input is offensive, the labeled transcription can be used.

Emotion may indicate response planning, like how we talk about responses-more subdued, validating, or maybe more excitingly, for instance. Not for sensitive decision making. Confidence-like value isn't safe without a calibrated model, and a model's softmax score/rule score not necessarily is probability.

2.8 Turn Taking, Voice Activity Detection, and Barge-In

Turn taking allows the assistant to know when the user has finished speaking and to return the response. For more basic interfaces, a record button can be used and the app waits for the user to release. For more complex ones, Voice Activity Detection (VAD) [29] can be used and pauses or thresholds are set for an approximation of an end of utterance. If there is other noise, VAD can be compromised or if the speaker is quiet and far away it can be inaccurate.

Barge-In (allow the user to interrupt the assistant while answering) In the current proposal the browser recognizes when there is a potential for Barge-in, pauses local playback, and kills the current turn, so that a new utterance can be recorded and understood. Pauses local playback, but there is no backend task pause support at this time (this should be considered for testing).

Timing may also be an issue. Response timing may be interpreted as a new turn. It is assumed that the use of the session ID and request ID will allow the system to determine what is a new turn. Barge-In testing should include detection time, cancellation time, response playtime, and acceptance of next utterance.

Chapter 3. Problem Definition and Objectives

3.1 Problem Definition

The work needs: To build a working voice assistant that can take the form of speech and text and return a spoken understanding response. An understanding of system state should be returned, enough so that if errors do occur they can be understood. The combination of transcription, emotion, language, text-to-speech and interruption canel are the key element here. The tests should clearly distinguish between request completion and speech recognition accuracy. It is recommended the score obtained from the model is not taken as a straightforward accuracy measure.

3.2 Objectives

The aim is to combine Speech recognition, Response generation, emotion and Speech to Text (STT) [27]. The demo should show the speech recognition, response generation, source of emotion, provider specific metadata, Speech to Text (STT) [27] and the response generated. Usage of the voice service should be in accordance with the instructions provided, including streamed audio [28] where supported by the provider pathway. Browser based VAD [29] should be used along with a cancel pathway to cancel the audio in the case of unsupported providers and cancel the request and to instruct the backend to stop the current turn. Evaluation should be based on example samples as described in the documentation with the numerators being clearly defined and all failures counted and the results at the case level traceable back to saved project files.

3.3 Scope and Out-of-scope

In scope	**Not established by this project**
Provider-based speech/text pipeline, emotion-policy integration, and research demo.	Training a new language, speech-recognition, speech-synthesis, or emotion model.
Small, documented pilots using public data sources.	Full SOVA-Bench reproduction or local execution of its published baseline models.
UI, captions, playback, and the code-level interruption path.	Guaranteed emotion accuracy, human MOS, or physical barge-in reliability.

3.4 Functional Requirements

Functional requirements describe the operations that the application is expected to provide. They are stated here as system capabilities, not as a claim that every possible browser, microphone, or provider configuration has been tested.

Table C1. Functional requirements for the emotion-aware voice assistant.

ID	Requirement	Expected system behavior
FR-01	Input capture	Accept a spoken request through the browser microphone when permission is granted.
FR-02	Text input	Allow a user to submit typed text through the same interaction interface.
FR-03	Transcription	Convert supported audio input into a text transcript using the configured speech-recognition service.
FR-04	Emotion estimation	Obtain an emotion cue from transcript rules and, when available, a compatible acoustic model.
FR-05	Response planning	Pass the selected emotion cue and response-style policy into response generation.
FR-06	Text response	Return the generated response text and associated response metadata.
FR-07	Speech output	Request speech synthesis from the configured provider and play the returned audio in the browser.
FR-08	Session handling	Associate interactions with a session and the authenticated caller.
FR-09	Interruption	Stop local playback and request cancellation of the active backend turn when barge-in is detected.
FR-10	Diagnostics	Expose relevant status and session metrics for demonstration and evaluation.

3.5 Non-Functional Requirements

Non-functional requirements describe qualities that influence how useful and dependable the application is. They are important for a browser-based voice system because an apparently small delay or unclear status can interrupt the conversational experience.

Table C2. Non-functional requirements and their relevance.

Quality	Requirement for the prototype	Reason
Responsiveness	The interface should show meaningful state changes while a turn is being processed.	The user needs to know whether the system is listening, thinking, or speaking.
Reliability	Provider and request failures should be surfaced without silently presenting an incomplete turn as successful.	External services and network conditions can vary.
Security	Voice routes should require valid authentication and the required permission.	Sessions and operations must be associated with the authorised caller.
Privacy	Credentials should remain in local configuration, and sensitive values should not appear in report material or logs.	The project uses external providers and user audio.
Maintainability	Speech recognition, generation, emotion processing, and synthesis should remain identifiable modules.	Modular boundaries support debugging and later replacement.
Observability	The application should make relevant state, model metadata, and error status inspectable.	This helps explain a result or diagnose a failed turn.
Compatibility	Browser audio features should be checked for permission and API support.	Microphone and speech-recognition behaviour is browser-dependent.

CHAPTER 4: METHODOLOGY & WORK DESCRIPTION

4.1 This method employed in this section implies a sequence of steps to generate an end-to-end turn.

First, the browser then makes a request that contains text or audio, turn, and session information. For request with audio, it would be probably transcribed using the speech recognition service in the configuration, and in case of text request, it is directly skipped. Then, the text from the transcribed message would be run through the emotion rules. Interestingly, for the compatible system's WAV format audio, it could also apply the acoustic emotion model.

In addition, the hybrid policy chooses an emotion label, source, and score. When no or incomplete acoustic data is available, a default response is generated. The emotion label and a short response policy are added to the request to the language model. The language model provides a possible response of both text and delivery details. At the output, and if a TTS configuration has been set up, the corresponding speech is streamed to the TTS provider [28] and the interface receives the text, response, emotion data, TTS provider [28] details, and turn duration.

4.2 Emotion Signal and Fallback

This emotion classification decision is an auxiliary signal. The acoustic checkpoint is enabled as required and expects resampled WAV audio 16kHz. Although the softmax scores are not scaled as confidence scores and the keyword rules contain constant scores as opposed to probabilities, these results should not be misconstrued as accuracy scores.

FIGURE 4.1 EMOTION-SIGNAL FUSION AND RESPONSE POLICY

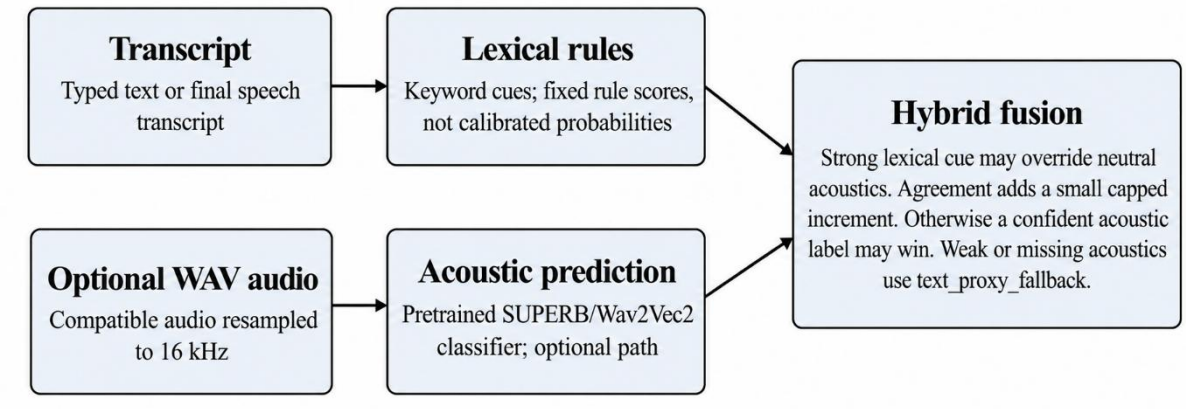


Figure 4.1 shows how lexical and acoustic signals are input into the response style policy.

If there are strong lexical cues then the acoustic cues will be ignored. There is also the option to give an additional boost if both cues agree, however there could also be a confident choice of acoustic cues. If the acoustic cues are low and not available the text cues will be used. The text cue source will be given, along with the emotion. The chosen cue type will influence the words used in the response. However there is no guarantee that the detected emotion is correct.

4.3 Audio input and transcription

Once the user has selected to interact the audio from the microphone device will be captured by the browser. Once permission has been given, the audio will be sent to the backend using the audio-turn route. The backend will check that the request isn't too large and that the content type is correct. If the audio file is larger than what is set in the route or can not be understood by the provider, reject it with the correct error.

The audio will be processed by the speech recognition. This will provide the transcript and could also return information about the language or how long the audio was. The transcript will be used by the assistant runtime as it is the language containing version. This is an important part as if words have not been picked up or misinterpreted by the speech recognition it will not be able to be retrieved in later stages. We should measure the quality of the transcript and whether the request was completed. Word Error Rates can be used to give us an indication of how different the recognition is and request coverage can tell us if we got a usable result from the service.

As the text is provided by the user we don't need to do any transcription. This should be noted in the experiment, just because we are not using the audio recognition doesn't mean we are doing a text only task.

4.4 Emotion Estimation and Fusion

We have two options for retrieving emotion information from the project. One option is to look at the transcript and use either keywords or rules to determine the emotion. If we have compatible WAV audio files we can use the pre-trained speech emotion classification model to get the emotion. This model wasn't trained for this project. The model will return some information which can be stored as model metadata. This could include a prediction and scores. As the model has not been calibrated we shouldn't assume the scores are probabilities.

The hybrid policy will decide which of the sources to use. If the text had strong lexical clues and the audio had weak clues we would prefer to use the text. If they both agree they can be used to provide context. If there is no prediction from the audio we can fall back to using the text and note that we used the text. This is useful as we could have the same emotion label from both sources.

We can use the emotion to change how we respond. If the user has said something frustrated or anxious we might want to respond in a calm way to validate their feelings. If they have said something positive we could use a more enthusiastic tone. However this is just about how we choose to communicate with them. We should still make sure our response is relevant to the request.

4.5 Response Generation and Text To Speech

Once we have created our emotion policy we will generate the request for generating the response. This will include the users transcript and a short instruction based on the chosen emotion clue. The response will be generated using the language model specified in the route. This will return some information about how the response should be expressed. The metadata returned can include emotion label, expression, gesture, gaze, lip sync mode, voice style, tokens used, provider/model name and the time taken to process. This can be used by the browser to show the results and by the operator to see what settings were used in the runtime.

We will then generate the text to speech. The text and voice settings will be passed to the text to speech provider [28]. The audio will be returned to the browser, this will depend on the route selected if it is returned as audio data or a stream. By having separate steps we can show the text response to the user even if the audio service isn't ready. We should make sure to note if we have received text only, audio or a stream.

4.6 Session and Interruption

A session can be used to provide a scope for a series of related turns. A session will be allocated to the authenticated caller by the backend. Each new request will include the session ID. The turn will be processed and information about the response will be returned. If the user interrupts whilst audio is playing the local audio will be stopped by the browser and a cancellation request will be made. The backend will try to cancel the current task.

It is important that we note if we have cancelled locally or on the server. We can stop the local audio straight away but the backend might not have finished. We might have a situation where the task has already been completed. A useful log entry would include information about the interruption, session ID, request ID and if we were able to cancel.

4.7 Error Handling

The voice API will return specific HTTP responses for common situations that occur. If there is no session or it can't be accessed we will return a not found. If there is a conflict with the current turn we can return a conflict. We can also differentiate between interrupted and completed turns. If the provider can't be used we will return a service unavailable. If the input for processing is invalid we will return a client error. If the audio is too big we will reject it.

We should make sure that we provide clear statuses to the browser. We shouldn't show the user that the audio has played successfully if we haven't received any. If the optional emotion model couldn't be warmed up we shouldn't show that we have a confident prediction from the acoustic model. Instead we should note the fallback we have used. We shouldn't count the request as a successful interruption until we have considered the client side stop and result from the backend.

CHAPTER 5 PROPOSED ALGORITHM

INPUT: text or audio request R with session and turn metadata.
1. Input the requested text/audio (R) along with session and turn information.
2. Check if the input is valid, rejecting empty, invalid, too large or unreadable requests.
3. If audio was provided then run the speech recognition service specified and extract a transcript from the audio. Else use the text provided.
4. Extract emotion cues from the transcript.
5. If valid WAV file is available, run the optional acoustic classifier and store classifier source and score.
6. Combine the results from any available sources. If the acoustic source is not strong or present use the labelled text proxy.
7. Add selected emotion and response style policies to the language model request.
8. Generate the text response along with delivery information.
9. Generate or stream audio. Return the transcript, text response, audio, emotion, service used and time information.
10. If browser VAD detects an interruption, pause the audio and cancel the current request. Request to cancel the session on the backend.

Output: Transcript, Text/Audio response from assistant, Emotion shown and Turn information.

5.1 Evaluation metric definitions

Metric	Formula	Meaning
Corpus WER (%)	100 × (S + D + I) / N	Word errors divided by reference words; input ASR and generated-speech consistency are separate tasks.
Coverage (%)	100 × successful requests / attempted requests	Execution completion, not recognition quality.
Answer rate (%)	100 × answered / attempted	Provider failures remain unanswered.
Conditioned accuracy (%)	100 × correct / completed and judged	Accuracy among returned and judged responses.
Semantic mean	sum of judge scores / judged outputs	Project-specific judge result; not the paper's exact hidden GPTScore protocol.

CHAPTER 6. PROPOSED FLOWCHART, BLOCK DIAGRAM AND DATA FLOW

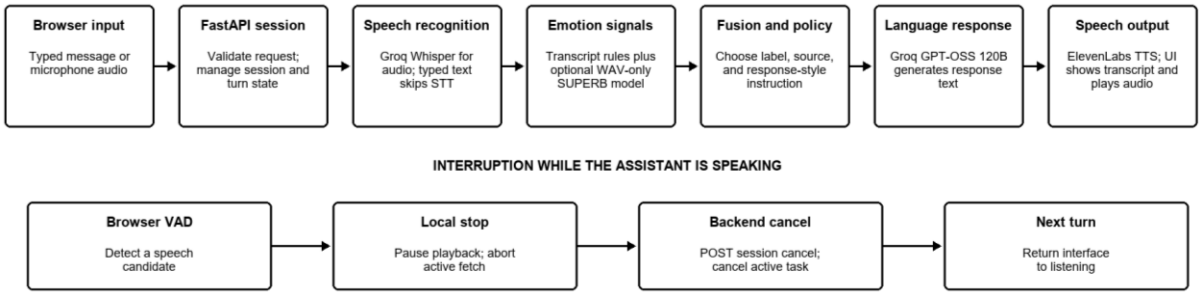


Figure 6.1 Architecture of turn-taking Emotion Aware Voice Assistant

The architecture shown in Figure 6.1 shows how information flows through a request from the browser, followed by a request via FastAPI [26], speech recognition [25], [27], emotion processing, language generation and finally speech synthesis [28]. The text input will bypass the speech recognition, this means that an acoustic classifier may be optional. In the audio playback, there is an option for the browser to use VAD [29] to stop the audio playing and cancel any current task running on the backend.

6.1 Pilot Evaluation Data Flow

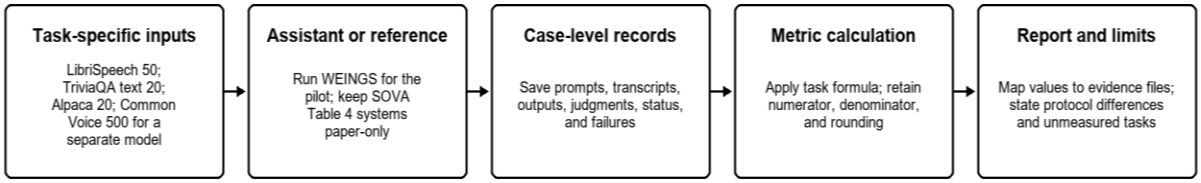


Figure 6.2 Evaluation and Evidence Flow

Each task has been assessed separately and cases have been systematically logged recording the number of tries and hits, text and audio outputs with decision points and problems encountered by the provider. All metrics are calculated with the formulas described in Chapter 8, using the values given in table 4 of SOVA-Bench as the benchmark. These are all paper based and have not been executed on the model locally.

6.2 Context Level

From the top of the system architecture, the browser application is used to interact with the user, and relays information to the FastAPI [26] backend which in turn interacts with the configured speech recognition [25], [27], language generation, emotion analysis and speech synthesis [28] components. Text and metadata are returned to the browser after a turn has been fully processed by the system. If audio is enabled, any audio data or stream is also returned. The browser then displays the transcript and response, updates the interaction state and may also send a request to terminate the interaction if interrupted by the user.

This diagram shows the main external actors involved and how they interact. Bear in mind that this is not a deployment diagram and that the speech, language and synthesis providers are likely to be 'hosted services'. The assistant backend itself runs inside a local environment that has been set up in this way. It has been implemented with well-separated modular service boundaries so that the backend can provide the operations that the providers need.

6.3 Level-1 Process Decomposition

Table E1. Level-1 data-flow processes.

Process	**Input**	**Processing responsibility**	**Output**
P1: Capture and validate	Microphone audio or typed text	Browser capture and backend request validation.	Accepted audio/text request or validation error.
P2: Speech recognition	Supported audio data	Configured STT [27] provider creates a transcript.	Transcript and available language/duration metadata.
P3: Emotion estimation	Transcript and optional compatible WAV input	Lexical rules and optional acoustic classifier produce candidate cues.	Selected emotion, source, and associated scores.

Process	Input	Processing responsibility	Output
P4: Response generation	Transcript and response-style policy	Configured language model generates response text and expression metadata.	Text response and runtime metadata.
P5: Speech synthesis	Response text and configured voice information	Configured TTS [28] provider generates audio or an audio stream.	Audio response or synthesis-unavailable status.
P6: Playback and interruption	Response text/audio and browser voice activity	Browser presents the response, plays audio, and may trigger local stop and server cancellation.	Completed or interrupted turn state.

6.4 Data Items Passed Between Stages

Table E2. Selected data items used in a voice turn.

Data item	Purpose	Reporting consideration
Session identifier	Associates several turns with one caller-owned session.	Use it to group session-level activity; do not expose authentication secrets.
Request identifier	Distinguishes an individual processing request.	Useful for matching logs and results to a single turn.
Transcript	Text recognised from audio or entered by the user.	The source and modality should be recorded.

Data item	Purpose	Reporting consideration
Emotion source	Identifies the lexical or acoustic path used.	Do not report the label without its source.
Emotion scores/top predictions	Shows the classifier or rule output available to the application.	Do not call uncalibrated scores probabilities or accuracy.
Response text	Contains the language model output.	Retain the exact output for case-level evaluation where permitted.
Provider/model metadata	Identifies the configured response-generation service.	Record safe model names only, never API keys.
Processing time and token usage	Supports operational analysis.	State whether the timing covers one stage or the complete turn.
Audio result and cancellation status	Describes synthesis/playback and interruption outcome.	Keep local playback stop separate from backend cancellation.

CHAPTER 7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

Table 2. Components identified in the project audit.

Layer	Component	Role and evidence boundary
Backend	FastAPI / Python [26]	Serves demo and voice endpoints in one application.
Speech recognition	Groq Whisper large-v3-turbo [25], [27]	Transcribes audio; used in the LibriSpeech input pilot.
Language model	Groq openai/gpt-oss-120b	Generates response text; provider settings can vary locally.
Acoustic emotion	superb/wav2vec2-base-superb-er [13]	Optional pretrained WAV classifier; no Emotion Aware Voice Assistant accuracy test set was run.
Lexical emotion	Keyword/rule detector	Transcript proxy; fallback is labelled text_proxy_fallback.
Speech synthesis	Configured ElevenLabs voice/model [28]	Generates spoken output; exact identifiers remain in local configuration.
Browser audio	Web Audio / Web Speech APIs [29], [30]	Supports audio analysis, VAD, and interim captions subject to browser support.
Interruption	Browser VAD and backend cancellation	Code path pauses playback and cancels the active task; physical behavior needs manual verification.

7.1 Backend & Configuration

• The backend for the demo and voice routes are a single FastAPI [26] application, which can be found under /api/v1.

• The API contains a number of endpoints such as health/diagnostic, session, text/audio turn, cancel, metrics, emotion readiness, transcription and synthesis.

• Provider credentials are stored locally in the .env file. No secrets have been provided within the report or evidence map.

• Provider and model information can be changed depending on the local configuration. The models listed in the report are audited and will not include provider credentials.

7.2 UI & Conversation State

Voice assistant has a UI that users can interact with the AI easily. The UI enables the users to either type to the AI assistant or use the microphone.
The UI displays the current mode of conversation. The icons displayed will be based on what state the assistant is currently. For example, Waiting, Listening, Processing, Generating, Speaking.

Conversation State
This makes sure the user interaction is correct. It makes sure that the conversation is still in sync between the frontend and the backend.

Here are a handful of states:
Idle - Waiting for new message from user.
Listening - Capturing the user voice.
Handling text /audio to backend -Speech recognizer will do recognition and Emotion analysis will also be done if necessary.

Generating - Language model produces a reply to the user input & conversation.
The response to the language model will be converted to speech and played to the user in the browser.

Interrupted - if the user is speaking when the assistant is talking, the audio will be interrupted and the assistant will cancel its speech. The UI will enter listening.

Error - Handles any unexpected errors.

Frontend & Backend Synchronisation
Both frontend and backend. When you submit a new message, the frontend will add it to the conversation then take the request to the backend. It will then process the request and return the response generated. This will be shown in browser and played.
One valuable aspect is the barge in. Should a user start talking whilst the assistant is speaking, the browser will pick this up. This will give the user back control, stopping the response, cancelling the current audio fetch, sending a cancel request to the backend to make sure the response that the user receives isn't stale.
The UI and conversation state work together seamlessly for a fluid interaction. Voice input, emotion aware processing, response generation, playing audio and barge-in all connect together.

7.3 Turn State & Captions

This demo includes a number of states including ready, listening, thinking, speaking, interrupted and completed. It also shows the transcript of the conversation and the response.

Captions will be shown while recording using the browser SpeechRecognition [30] tool if available. This will only be the case while recording and the backend transcript will always be definitive. There is also an option to press record/finish to stop a long utterance. Whether the microphone has access, a device is selected and captions are available will depend on the environment the demo is running in.
The emotion shown is metadata and therefore not definitive. The text_proxy_fallback shows that the rules are being applied based on the transcript. The voice used for synthesis is the one specified in the configuration. The exact name of the voice and model will be based on the local configuration.

7.4 Barge-in Path

Stage	Implemented action	Evidence boundary
Playback	Browser plays assistant audio.	Historical provider smoke recorded a successful audio response.
Detection	Browser VAD detects a speech candidate.	Microphone permission and physical behavior need an in-browser check.
Local stop	Pause playback and abort active fetch.	Implemented in the client; latency is browser-side.
Backend cancel	Cancel active session task and mark the turn interrupted.	Physical end-to-end barge-in was not verified in the saved smoke.

7.5 Reproducibility

It is suggested that you make sure that the.env file is kept secret. Researchers should keep track of secure provider and model names in their notes of the experiment. In the beginning of a new experiment, you should record the source revision, package lock, manifest, provider or model configuration, and the command used to run the experiment. In addition, it is suggested that you re-calculate from output- and formula files instead of copying the results and summarizations.

7.6 Voice API Surface

The voice API consists of various routes located under the /voice router. It includes the following routes: session, text, audio, streaming audio, cancel, metrics, emotion model status, model warm-up, and transcription. The voice endpoints are probably being called by the client with a bearer token Authentication. The following short overview is a listing of route groups available in the project source: Please note that this implementation tutorial acts as an auxiliary material and should by no means be a substitute for the API schema.

Table F1. Selected voice API routes in the implementation.

HTTP method and route	Purpose	Relevant output or effect
POST /voice/sessions	Create a session for the authenticated caller.	Returns a session identifier and initial status.
POST /voice/sessions/{session_id}/turns	Process a transcript supplied as text.	Returns response text, expression metadata, emotion information, token usage, provider/model, and processing time.
POST /voice/sessions/{session_id}/audio-turn	Process one audio turn through speech recognition, runtime, and synthesis.	Returns audio response and safe turn metadata.
POST /voice/sessions/{session_id}/audio-stream-turn	Process an audio turn using a streaming response path.	Returns a stream that can be interrupted by the client.
POST /voice/sessions/{session_id}/cancel	Request cancellation of the caller's active turn.	Returns a cancellation boolean.
GET /voice/sessions/{session_id}/metrics	Read session-level research metrics.	Returns active status and interruption count.
GET /voice/emotion/status	Read acoustic emotion-model readiness information.	Reports the available model status.
POST /voice/emotion/warmup	Request loading of the optional acoustic emotion model.	Returns readiness information and may expose a warm-up error.
POST /voice/transcriptions	Transcribe raw audio without saving the source audio bytes to disk in this route.	Returns transcript and available language/duration metadata.

7.7 Request and Response Metadata

A text turn request will contain a transcript and an optional language field. The transcript will have been validated to ensure it has been provided and that it is not longer than the maximum transcript length specified in the request schema. Metadata in the response will contain the session ID and request ID to link this turn to the correct session and log. The text generated will be provided, along with instructions to generate the expression, an estimated emotion prediction, a confidence like measure, the source of the text, a list of top emotion predictions, number of tokens used, the provider/model name and the time taken for processing. Acoustic and lexical detail objects can be provided to give more detail about the chosen emotion path.

This metadata can help during a demonstration to show the reasoning behind the assistant's choice of expression. During evaluation, it can help associate the output with the model configuration and input example. However, the presence of a confidence value does not mean that the emotion prediction is well calibrated and the presence of a provider/model name does not mean the model has been trained/fine-tuned using data from the evaluation dataset.

7.8 Authentication and Session Association

Bearer Authentication will be applied to the voice routes and therefore a check will be made to ensure the caller has the correct VOICE permission. The service can then retrieve the authenticated subject and this will be used when creating/accessing sessions. This ensures that there is a link between the caller and session. This also ensures that the caller can not use another user's session ID. Internal session and processing error codes will also be mapped to safe HTTP responses within the route layer.

The provider credentials will be retrieved from local configuration and should not be included in any source code, screenshots, report tables or demonstration logs. The report should only mention the providers and model names if it is safe to do so. Any experiment that needs to be reproducible can include a configuration name or source revision however secret values should never be included.

7.9 Debugging

During a live demonstration, the debugging should focus on showing what is happening during the interaction and what has happened during each processing step. This can include information such as if a session exists, if the optional acoustic model has been initialised, if the speech recognition has provided a transcript, what emotion source has been used, what response model has been called, if the synthesis has provided audio and if any cancellation requests have impacted on an active turn. This will make the demonstration easier to explain and any failures easier to categorise.

The debugging panel should not be seen as a measure of performance. If the provider is shown as ready this just means that the check has passed for that point in time. It will not indicate that the transcription, emotion classification, semantic accuracy or natural audio has been achieved. These factors will need to be measured using task specific datasets as outlined in the result analysis chapter.

CHAPTER 8. RESULT DISCUSSION AND ANALYSIS

The evaluation uses fixed local samples and is not a full SOVA-Bench reproduction. The paper's model rows and Cascade are reference values transcribed from the supplied paper

Table 3. Emotion Aware Voice Assistant evaluation metrics, denominators, and limitations.

Task	Data / sample	Local result	Calculation / caveat
Speech recognition	LibriSpeech; 1,000 utterances, 250 per split	3.55% corpus WER; 100% request coverage	656 substitutions, deletions and insertions / 18,466 reference words; all 1,000 requests completed.
Knowledge QA	TriviaQA; 55 text questions	0.00% exact-match proxy; 35.58% mean token F1; 100% answered	Exact match and token F1 are lexical proxies; the paper's GPT evaluator was not used.
Spoken QA	LibriSQA Part II; 55 selected cases	74.55% all-case accuracy; 78.85% conditioned accuracy	41/55 correct; 52/55 valid options; conditioned denominator 52.
Spoken QA	Spoken SQuAD; 55 audio questions	0.00% exact-match proxy; 25.54% mean token F1	All 55 backend responses returned; lexical proxies, not paper GPT-evaluated accuracy.
Text generation	Alpaca; 55 prompts	20.93% mean ROUGE-L F1; 26.66% mean token F1	Text-only lexical proxies; no GPTScore, audio consistency, or acoustic score.
Emotion understanding	IEMOCAP	Not measured	Dataset unavailable; no labelled held-out emotion result.
Age / gender	Common Voice	Not measured	Age and gender are not assistant outputs.
Evaluation total	1,000 ASR + 220 QA/generation cases	1,220 evaluated cases	Five task families; fixed samples do not reproduce the full SOVA-Bench protocol.

8.1 Experiments not performed

• Accuracy on the emotion detection task with labelled unseen audio.
• The accuracy of the TriviaQA and Spoken SQuAD datasets reported in this paper, using GPT to evaluate the responses. The full SOVA-Bench scores are also available.
• Listeners' MOS or UTMOSv2 scores for the quality of generated audio samples.
• Full task splits used for SOVA-Bench, full results for the Alpaca evaluation on all 3745 prompts and the baseline models described in this paper run locally.

8.2 SOVA-Bench published recognition and understanding values

Table 4(a) reproduces the supplied paper's published values [1]. These are literature references; the baseline systems were not run locally.

Table 4(a). Published recognition and understanding metrics from SOVA-Bench.

Model	LLM size	Know-ledge	Dev clean / other WER	Test clean / other WER	LibriSQA	Spoken SQuAD	Emot-ion	Age	Gender
Mini-Omni	0.5B	2.42	5.18 / 12.03	8.68 / 12.75	—	—	28.53	11.79	27.22
Mini-Omni2	0.5B	0.37	7.54 / 11.58	3.99 / 13.44	—	—	24.52	16.05	16.73
Moshi	7B	4.00	—	—	—	—	—	—	—
LLaMA-Omni	8B	24.91	>100 / >100	>100 / >100	50.67	43.07	37.74	19.43	78.41
Freeze-Omni	7B	27.27	3.29 / 7.40	3.24 / 7.68	—	—	38.31	17.80	39.13
GLM-4-Voice	7B	25.58	—	2.82 / 7.66	—	—	30.16	37.84	69.40
Cascade	—	58.98	1.92 / 3.77	1.81 / 3.62	81.11	93.05	—	—	—
Emotion Aware Voice Assistant (local)	Groq GPT-OSS 120B	0.00 exact-match proxy (n=55)	2.75 / 4.79 (dev; n=250 each)	2.51 / 4.52 (test; n=250 each)	74.55 (n=55)	0.00 exact-match proxy (n=55)	Not mea-sured	Not app-lic-able	Not appl-icable

WER is lower-is-better; other reported task scores are higher-is-better. A dash means the paper does not report a value. The ">100" entries are retained as printed.

8.4 Local interpretation

• LibriSpeech Word Error Rate (WER) is computed by taking the micro-average across 18,466 words in the reference text for 1000 utterances (250 words per split). All 1000 requests were completed.

• The TriviaQA answer coverage metric has been included alongside lexical overlap. This ensures that all cases have been captured in the denominator. The exact match and token F1 metrics are proxy metrics for the true accuracy of the model, which was measured using GPT in the paper.

• Alpaca results do not include any human evaluation or GPTScore results collected according to the protocol in the paper. They were instead evaluated using text only ROUGE L and token F1 metrics.

• Age and gender are not output by the assistant and therefore were not evaluated in this run.

8.5 Local and paper protocol mapping

Table 5. SOVA-Bench tasks and local evaluation coverage.

SOVA-Bench task	Paper dataset size	Local evaluation / status	Comparability
Knowledge QA: TriviaQA	6,840	55 questions; 55 answered; exact-match proxy 0.00%, token F1 35.58%	Lexical proxies; not paper GPT-evaluated accuracy.
Speech recognition: LibriSpeech	Four evaluation sets	1,000 utterances across four splits; corpus WER 3.55%; 1,000/1,000 completed	250 per split; adapted sample, not full evaluation sets.
Spoken QA: LibriSQA	2,619	55 cases; 41 correct; 52 valid options; 74.55% all-case and 78.85% conditioned accuracy	Cascade-style use of saved ASR transcripts.
Spoken QA: Spoken SQuAD	3,243	55 audio questions; exact-match proxy 0.00%, token F1 25.54%	Lexical proxies; not paper GPT-evaluated accuracy.
Emotion: IEMOCAP	5,819	Not measured	Dataset unavailable; no assistant emotion accuracy.
Age and gender: Common Voice	2,511 age; 2,469 gender	Not measured	These are not outputs of this assistant.
Generation: Alpaca	3,745	55 text prompts; ROUGE-L F1 20.93%, token F1 26.66%	Text-only lexical proxies; not GPTScore or speech metrics.
Overall scope	Full SOVA-Bench	1,220 cases across five task families	Fixed local samples; published baseline checkpoints were not rerun.

8.6 Examples of calculations

• Corpus WER = 100 x (S+D+I)/N. For LibriSpeech, this is 100 x 656 / 18,466 = 3.5525. (Reported as 3.55%) This is an aggregate result across 4 different splits, and individual results are dev-clean: 2.75%, dev-other: 4.79%, test-clean: 2.51% and test-other: 4.52%.

• The answer coverage for the LibriSpeech request is 1,000/1,000 x 100 = 100%. This measures how many requests have been completed.

• The answer coverage for TriviaQA is 55/55 × 100 = 100%. The average token F1 is 35.5836%. The exact match proxy is 0/55 = 0.00%. The GPT evaluator described in the paper was not used.

• The accuracy for LibriSQA is 41/55 × 100 = 74.5455%. This is 41/52 × 100 = 78.8462% when conditioning on 52 valid answer choices. Alpaca had an average ROUGE-L F1 score of 20.9270% and an average token F1 score of 26.6647% across 55 pieces of text.

8.7 Limitations & Threats to Validity

• The samples used for knowledge and generation were of a limited size and cannot infer performance across the whole source dataset. Also,

• Local protocols differed from the methods described in the paper. Different sample sizes, modalities, providers/models, and prompts for judging were used. The results aren't a direct measure of performance comparing to the published systems.

• All 1000 LibriSpeech requests and sampled questions/prompts (220) returned responses from the backend. However, 3 LibriSQA outputs had no valid option and were therefore part of the overall denominator. They will not be included in the conditioned accuracy.

• The emotion accuracy wasn't measured using a labelled hold out set. The acoustic softmax and fixed lexical rule scores aren't real accuracies and won't be calibrated.

• There was no measurement of the quality of the generated speech, WER of consistent transcripts, MOS by humans, and UTMOSv2.

• It will depend on the hosted services and local setup. Also,

VAD was performed in the browser and cancellation in the backend. The smoke file doesn't confirm that a person was speaking into the browser microphone, or end-to-end barge-in. There were 32 focused tests successful in the smoke file saved. In the full suite there were 661 successful, 52 failed and 22 errors. This is a snapshot from 15 October 2026.

8.8 Validation strategy

Component testing: Testing individual functions within the application, such as validating requests, processing transcripts, choosing appropriate emotion fallbacks, calculating metrics etc.

Integration testing: Ensuring data from one module is accepted by another.

End-to-end testing: Using the microphone and speakers on the browser to test the whole spoken turn. There are 3 distinct levels of testing here and they should be treated as separate.

Automated tests can test deterministic behavior such as validating responses use the correct schema, ensuring sessions have the correct ownership, mapping errors correctly, using the correct metric calculation and validating the state changes for cancellation however it can't guarantee that a real mic will pick up the users voice reliably or that the browser will stop playing audio fast enough if the user interrupts. This needs to be tested using the browser with permissions for the mic and speakers.

8.9 Reproducibility

To ensure meaningful results from the benchmark it is important to know exactly what inputs were provided, what configuration was used and how the scores are calculated. Every run should include details such as the date of the experiment, version of the source, name of the dataset, split and seed (either manifest or sample selection), number of cases attempted, number of cases completed, number of cases failed, provider/model and metric implementation. If there has been any judgement carried out the model used by the judge, prompt/rubric and handling of partial outputs should be included.

Output files at the case level should be saved so that the reviewer can look at individual transcripts, model outputs, error cases and see how the scores were calculated. Percentages should always have the denominator included. If a percentage is being calculated based on attempted cases then this should not be referred to as accuracy. If it is being calculated based on only correctly answered and judged cases then this should be stated.

Public repository (source code, run instructions and shareable evaluation summaries):
https://github.com/Rohit-Seera/Emotion-Aware-VoiceAssistant

8.10 Metric interpretations and limits

Word Error Rate is calculated using substitutions, deletions and insertions divided by the number of words in the reference transcript. Corpus WER should be calculated by summing all errors and reference words in the set of data you are evaluating. It shouldn't be calculated by taking an average of the percentage error per utterance. Request coverage is the percentage of requests that were completed out of those attempted. This is not a measure of recognition accuracy but how much of the service was completed.

For question answering tasks answer rate and accuracy should be reported separately. If a lexical metric such as exact match or token F1 is being used to represent the model performance then it should be noted that this is not the score achieved by the human or model judge for the benchmark. For generation tasks text overlap metrics do not measure how natural the generated speech is. To make claims about speech quality human listening scores or specific acoustic metrics should be used. The output of an emotion classifier on unlabelled interactions cannot be used to calculate emotion classification accuracy. The runs reported have been carried out using sample data and modified processes. This means they cannot be fully reproduced using SOVA-Bench. The baseline results published will still be referenced in the paper unless the relevant checkpoints have been run locally using the same protocol [1].

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

• The project demonstrates how to map text and speech inputs to an emotion aware text/speech output using FastAPI [26]. The demo includes information on the text transcript, emotion source, generated text, play status and cancellation route. The evaluation includes how well the tasks of recognition, question answering and text generation were completed

• It provides specific information about the performance including how many cases were completed, the word error rate during recognition, how many answers were provided, the accuracy of the answers, and lexical overlap. This was done using the SOVA-Bench to provide context, however there were varying numbers of samples and protocols meaning the local results are not a direct comparison to the published systems.

• There is still a need to evaluate the emotion using a labelled dataset. There will also need to be human evaluation of the audio output and multiple attempts at capturing interruptions from the browser microphone to form conclusions around the quality of the audio.

9.2 Future Scope:

• Evaluate emotion using a labelled dataset. Report precision, recall, macro F1 score and calibration per class.

• Use agreed subsets of the full SOVA-Bench with fixed providers, manifests and clear reporting of failures.

• Include a controlled listening test and an acoustic quality metric.

• Evaluate interruption detection, delay until stop playing, and cancellation success using multiple attempts at capturing interruptions from the browser microphone.

• Ablation study into text only, audio only and combined emotion policies.

REFERENCES

[1] Y. Hou, H. Liu, Y. Wang, Z. Cheng, R. Wu, Q. Gu, Y. Wang, and Y. Wang, "SOVA-Bench: Benchmarking the Speech Conversation Ability for LLM-based Voice Assistant," in Proc. Interspeech 2025, pp. 5713–5717, doi: 10.21437/Interspeech.2025-1950.

[2] Z. Xie and C. Wu, "Mini-Omni: Language models can hear, talk while thinking in streaming," arXiv:2408.16725, 2024.

[3] Q. Fang et al., "LLaMA-Omni: Seamless speech interaction with large language models," arXiv:2409.06666, 2024.

[4] A. Défossez et al., "Moshi: a speech-text foundation model for real-time dialogue," arXiv:2410.00037, 2024.

[5] C.-Y. Huang et al., "Dynamic-SUPERB: Towards a dynamic, collaborative, and comprehensive instruction-tuning benchmark for speech," in Proc. ICASSP, 2024, pp. 12136–12140.

[6] C.-Y. Huang et al., "Dynamic-SUPERB Phase-2: A collaboratively expanding benchmark for measuring the capabilities of spoken language models with 180 tasks," arXiv:2411.05361, 2024.

[7] B. Wang et al., "AudioBench: A universal benchmark for audio large language models," arXiv:2406.16020, 2024.

[8] Q. Yang et al., "AIR-Bench: Benchmarking large audio-language models via generative comprehension," arXiv:2402.07729, 2024.

[9] Y. Chen, X. Yue, C. Zhang, X. Gao, R. T. Tan, and H. Li, "VoiceBench: Benchmarking LLM-based voice assistants," arXiv:2410.17196, 2024.

[10] C. Tang et al., "SALMONN: Towards generic hearing abilities for large language models," in Proc. ICLR, 2024.

[11] D. Zhang et al., "SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities," in Findings of EMNLP, 2023, pp. 15757–15773.

[12] Y. Chu et al., "Qwen2-Audio technical report," arXiv:2407.10759, 2024.

[13] W.-N. Hsu et al., "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 29, pp. 3451–3460, 2021.

[14] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," in Adv. Neural Inf. Process. Syst., vol. 33, 2020, pp. 17022–17033.

[15] X. Wang et al., "Freeze-Omni: A smart and low latency speech-to-speech dialogue model with frozen LLM," arXiv:2411.00774, 2024.

[16] Z. Xie and C. Wu, “Mini-Omni2: Towards open-source GPT-4o with vision, speech and duplex capabilities,” arXiv:2410.11190, 2024.

[17] A. Hurst et al., “GPT-4o system card,” arXiv:2410.21276, 2024.

[18] C.-H. Lee, S.-L. Wu, C.-L. Liu, and H.-Y. Lee, “Spoken SQuAD: A study of mitigating the impact of speech recognition errors on listening comprehension,” in Proc. Interspeech, 2018, pp. 3459–3463.

[19] C. Busso et al., “IEMOCAP: Interactive emotional dyadic motion capture database,” Language Resources and Evaluation, vol. 42, no. 4, pp. 335–359, 2008.

[20] Z. Zhao et al., “LibriSQA: A novel dataset and framework for spoken question answering with large language models,” IEEE Trans. Artificial Intelligence, 2024.

[21] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in Proc. ICASSP, 2015, pp. 5206–5210, doi: 10.1109/ICASSP.2015.7178964.

[22] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” in Proc. ACL, vol. 1, 2017, pp. 1601–1611, doi: 10.18653/v1/P17-1147.

[23] R. Ardila et al., “Common Voice: A massively-multilingual speech corpus,” in Proc. LREC, 2020, pp. 4218–4222.

[24] R. Taori et al., “Stanford Alpaca: An instruction-following LLaMA model,” project report and dataset, 2023. [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca

[25] A. Radford et al., “Robust speech recognition via large-scale weak supervision,” in Proc. ICML, PMLR, vol. 202, 2023, pp. 28492–28518.

[26] FastAPI, “Tutorial — User Guide.” [Online]. Available: https://fastapi.tiangolo.com/tutorial/. Accessed: Oct. 29, 2026.

[27] Groq, “Speech to Text.” [Online]. Available: https://console.groq.com/docs/speech-to-text. Accessed: Oct. 29, 2026.

[28] ElevenLabs, “Streaming text to speech.” [Online]. Available: https://elevenlabs.io/docs/eleven-api/guides/how-to/text-to-speech/streaming. Accessed: Oct. 29, 2026.

[29] Mozilla, “AnalyserNode — Web Audio API.” [Online]. Available: https://developer.mozilla.org/docs/Web/API/AnalyserNode. Accessed: Oct. 29, 2026.

[30] Mozilla, “SpeechRecognition — Web Speech API.” [Online]. Available: https://developer.mozilla.org/docs/Web/API/SpeechRecognition. Accessed: Oct. 29, 2026.